

Time Series Classification Assignment - Workflow Summary

Prepared by Prachi Dhurve | CoStrategix Technologies - Member Data Scientist assignment

This document summarizes the end to end workflow used in the notebook submission. The goal of the assignment was to discover repeating patterns in class 1 sequences, convert those patterns into meaningful features, train a classification model, and evaluate whether the solution could be useful in a real setting.

1. Problem setup and data understanding

The input feature file contained time series samples with shape **(batch, 10, 2)**. This means each sample has 10 timesteps and 2 numerical observations at each timestep.

The target file contained binary labels. Class 0 represented the majority pattern and class 1 represented the rare pattern that needed to be discovered.

A missing value check was performed using **np.isnan()**. No missing values were found, so no imputation step was required.

No label encoding or one hot encoding was needed because all variables were already numerical.

2. Class imbalance check

The dataset was found to be extremely imbalanced. Class 0 accounted for about 99.93% of the data, while class 1 accounted for only about 0.07%. Because of this, accuracy was not treated as a meaningful performance metric. The evaluation focused on recall, F1 score, ROC-AUC, and PR-AUC.

Class	Approx. count	Approx. percentage
0	999,298	99.93%
1	693	0.07%

3. Exploratory analysis and pattern discovery

Individual sequences from both classes were plotted first to understand the raw temporal behavior of the two signals.

Mean pattern plots showed that class 1 sequences had a clear **valley shaped dip in the middle timesteps**, roughly around steps 3 to 6. Class 0 sequences were much flatter and more stable.

Standard deviation plots showed that class 1 had stronger variability, which suggested a more structured and dynamic signal.

First order temporal differences were then computed using **np.diff()**. This confirmed that class 1 had sharper transitions entering and leaving the middle dip region.

Main pattern discovered: class 1 sequences show a distinctive middle region dip, along with stronger temporal transitions and higher variability than class 0.

4. Feature engineering

Since the final models were classical machine learning models and not sequence models, the time series had to be converted into tabular features. The engineered features were designed directly from the observed temporal pattern.

Feature group	Purpose
Global statistics	Mean, standard deviation, minimum, maximum, and range for each signal
Middle window features	Capture the dip pattern around timesteps 3 to 6 using mid mean and mid minimum
Temporal difference features	Capture sharp transitions using difference mean, difference standard deviation, and largest jump
Sequence shape features	Compare first half and second half behavior and approximate overall trend
Cross feature relationship	Measure the gap between feature 1 and feature 2 overall and in the middle window

5. Train test split and model training

The engineered feature table was split into training and test sets using an 80-20 split with **stratify=y** so that class imbalance was preserved in both sets.

Two models were evaluated. Logistic Regression was used as a baseline model and Random Forest was used as the main non linear model.

Class weighting was used in both models to handle the rare positive class. Logistic Regression used **class_weight='balanced'** and Random Forest used **class_weight='balanced_subsample'**.

6. Model selection

To reduce randomness in metrics and make model comparison more stable, **Stratified K-Fold Cross Validation** was used on the training data. The main metrics tracked were precision, recall, F1 score, ROC-AUC, and PR-AUC. Random Forest performed better overall and was selected as the final model.

7. Final model results

Metric	Final model result	Interpretation
ROC-AUC	about 0.95	Strong ranking ability between positive and negative classes
PR-AUC	about 0.028	Small in absolute terms, but much better than the random baseline for this rare-event setting
Recall	about 0.72	The model captures most class 1 events
Precision	about 0.008	Many false positives remain due to extreme class imbalance

The confusion matrix showed that the model detected a large portion of the rare positives, but it also produced many false positives. That tradeoff is common in rare event detection problems.

8. Feature importance and reasoning behind the model

Feature importance from the Random Forest model showed that the model relied heavily on variability features, middle window features, and temporal change features.

Examples of important features included **f1_std**, **f2_mid_min**, **f1_diff_std**, **f2_mid_mean**, **f1_mid_min**, and **overall_gap_mean**.

This matched the exploratory analysis and confirmed that the model was using the same temporal pattern that was identified visually earlier.

9. Production readiness and limitations

The model shows that the discovered temporal features are genuinely useful, so the solution has value as a rare pattern detection approach.

However, it should not be treated as a production ready model without additional monitoring because precision is still low and the false positive rate is high.

If new samples contain temporal patterns that were not present during training, the model may not generalize well. In a real environment this would require monitoring, retraining, and possibly refreshed feature engineering.

10. Final conclusion

Overall, the notebook followed a complete machine learning workflow: understand the data, discover the class 1 pattern, convert that pattern into engineered features, compare models with cross validation, evaluate on a held out test set, and interpret the results. The strongest takeaway is that the model did not rely on random correlations. It relied on a clear middle region dip, stronger temporal changes, and higher variability in class 1.

Submitted deliverables: Jupyter Notebook (.ipynb), workflow summary PDF, and GitHub repository link.